ATPG and Fault Coverage Analysis of a CRC-8 Integrity Module

Course: ECS324 VLSI Testing & Verification

Team Members:
Mukund Rathi [2023BEC0051]
Aniketh Yuvaraj C. [2023BEC0049]

Faculty Mentor:
Dr. Lakshmi N S

February 23, 2026

Contents

1 Objective & Overview **1**
1.1 Objective . . . 1
1.2 Overview of CRC-8 Architecture . . . 2

2 Theoretical Background **2**
2.1 Single Stuck-At Fault (SSF) Model . . . 2
2.2 Fault Equivalence & Collapsing . . . 2
2.3 Logic Masking & Redundancy . . . 2
2.4 Intentional Hardware Redundancy: The 9th Bit (Error Flag) . . . 2

3 Project Workflow & Methodology **3**

4 Phase 1: RTL Design & Simulation **4**
4.1 Verilog RTL Code (`crc8.v`) . . . 4
4.2 Testbench Code (`tb.v`) . . . 4
4.3 Tool Usage [Icarus Verilog GTKWave] (`bash`) . . . 4
4.4 Simulation Results . . . 5

5 Phase 2: Logic Synthesis & Fault Calculation **5**
5.1 RTL Schematic . . . 5
5.2 Theoretical Fault Calculation . . . 6
5.3 Generated Test Patterns (`.test`) . . . 6
5.4 Undetected Fault List (`.ufaults`) . . . 7

6 Phase 3: Formal Equivalence Checking **7**

7 Phase 4: ATPG Bench Generation (Python Automation) **8**

8 Phase 5: ATPG Execution & Fault Analysis **9**
8.1 Baseline CRC-8 Architecture (100% Testability) . . . 9
8.2 Modified Architecture with Hardware Redundancy . . . 10

9 Manual Verification of Redundant Faults (Hand Trace) **10**

10 Conclusion **12**

1 Objective & Overview

1.1 Objective

The objective of this project is to implement a synthesizable 8-bit Cyclic Redundancy Check (CRC) generator and evaluate its testability using Automatic Test Pattern Generation (ATPG). We aim to establish a baseline of 100 percent testability, and subsequently inject intentional hardware redundancy to demonstrate logic masking and redundant fault identification.

1.2 Overview of CRC-8 Architecture

Cyclic Redundancy Check (CRC) is an error-detecting code used to detect accidental changes to raw data. The ATM-8 polynomial used in this project is $x^8 + x^2 + x + 1$. While typically implemented using a sequential Linear Feedback Shift Register (LFSR), this project utilizes a parallel, purely combinational XOR-tree.

By mathematically unrolling 8 clock cycles of LFSR shifts into a single combinational layer, the hardware calculates the 8-bit remainder instantaneously. Because it lacks sequential elements (flip-flops), it is an ideal candidate for demonstrating the D-Algorithm and FAN ATPG algorithms without requiring scan-chain insertion.

2 Theoretical Background

2.1 Single Stuck-At Fault (SSF) Model

The SSF model assumes that a manufacturing defect causes a single node in the circuit to be permanently tied to Logic 0 (Stuck-at-0) or Logic 1 (Stuck-at-1). ATPG tools attempt to find an input vector that sensitizes the fault and propagates the error to an observable primary output.

2.2 Fault Equivalence & Collapsing

To reduce the computational complexity of ATPG, faults are collapsed. Two faults are equivalent if any test pattern that detects one also detects the other. For example, in an AND gate, a Stuck-at-0 on any input is equivalent to a Stuck-at-0 on the output. By collapsing these, the tool reduces the total number of simulated faults.

2.3 Logic Masking & Redundancy

A fault is considered "redundant" if no possible input vector can propagate its effect to a primary output. This occurs when boolean logic creates a masking effect. In this project, we utilize the Consensus Theorem ($F = AB + A'C + BC$) where BC is the redundant consensus term. We predict that faults on the BC path will be completely masked by the AB and $A'C$ paths.

2.4 Intentional Hardware Redundancy: The 9th Bit (Error Flag)

Why add a 9th bit?
The primary CRC-8 generator is a purely combinational XOR-tree. Because it lacks deeply nested reconvergent fanouts, ATPG tools effortlessly achieve 100% fault coverage with zero redundant faults. While 100% coverage is ideal in commercial manufacturing, it provides no opportunity for academic fault analysis. To demonstrate advanced VLSI testing concepts like logic masking and untestability, we intentionally injected a flawed, mathematically redundant sub-circuit into the design.

What is the Error Flag?
We introduced a 9th primary output named `error_flag`. This output does not interfere with the core CRC remainder calculation. Instead, it taps into the first three data inputs

($A =$ `data_in_0`, $B =$ `data_in_1`, $C =$ `data_in_2`) and processes them through a custom logic block based on the **Consensus Theorem**:

$$F = AB + A'C + BC$$

How does it work (The Hazard Cover)?
By rules of Boolean algebra, the BC term (the consensus term) is mathematically redundant; the equation safely simplifies to $F = AB + A'C$. However, in physical VLSI design, the BC gate is often intentionally hardcoded into the layout as a **Static-1 Hazard Cover**.

If variable A transitions from 1 to 0 while $B = 1$ and $C = 1$, the AB gate turns off and the $A'C$ gate turns on. During this tiny microsecond of propagation delay, both gates might output 0, causing the final output to momentarily glitch. The BC gate prevents this glitch by staying solidly at 1 during the entire transition.

How is this helpful for ATPG Analysis?
Because the BC gate is functionally redundant, any stuck-at fault occurring specifically on the BC path is fundamentally **untestable**. If the ATPG tool attempts to sensitize a fault on the BC gate by forcing its inputs ($B = 1, C = 1$), the other parallel gates (AB or $A'C$) will automatically evaluate to 1 (since A must be either 0 or 1). This parallel 1 overpowers the final output, completely masking the fault.

By adding this 9th bit, we intentionally lay a trap for the Atalanta ATPG tool. It forces the tool to encounter Logic Masking, drops the fault coverage slightly below 100%, and provides the exact redundant faults necessary to perform a manual D-Algorithm hand-trace.

3 Project Workflow & Methodology

The verification and testing methodology utilizes an open-source VLSI toolchain:

1. **RTL Design & Simulation:** Verify functional correctness using `iverilog` and `GTKWave`.

2. **Logic Synthesis:** Generate a gate-level netlist using `Yosys` and `Vivado`.

3. **Formal Verification:** Conduct Equivalence Checking (EQY) to ensure netlist-RTL parity.

4. **ATPG Python Automation:** Develop a custom Python script to mathematically map XOR logic into ISCAS89 `.bench` NAND formats, bypassing legacy parser limitations.

5. **Fault Simulation:** Run `Atalanta` and analyze the identification of detectable vs. redundant faults.

6. **Manual Verification:** Perform a manual D-algorithm hand-trace of the redundant logic to mathematically prove untestability.

4 Phase 1: RTL Design & Simulation

4.1 Verilog RTL Code (crc8.v)

```
module crc8 (
    input [7:0] data_in,
    input [7:0] crc_in,
    output [7:0] crc_out
);
    assign crc_out[0] = data_in[7] ^ data_in[6] ^ data_in[0] ^ crc_in[0] ^ crc_in[6] ^ crc_in[7];
    assign crc_out[1] = data_in[6] ^ data_in[1] ^ data_in[0] ^ crc_in[0] ^ crc_in[1] ^ crc_in[6];
    assign crc_out[2] = data_in[6] ^ data_in[2] ^ data_in[1] ^ data_in[0] ^ crc_in[0] ^ crc_in[1] ^ crc_in[2] ^ crc_in[6];
    assign crc_out[3] = data_in[7] ^ data_in[3] ^ data_in[2] ^ data_in[1] ^ crc_in[1] ^ crc_in[2] ^ crc_in[3] ^ crc_in[7];
    assign crc_out[4] = data_in[4] ^ data_in[3] ^ data_in[2] ^ crc_in[2] ^ crc_in[3] ^ crc_in[4];
    assign crc_out[5] = data_in[5] ^ data_in[4] ^ data_in[3] ^ crc_in[3] ^ crc_in[4] ^ crc_in[5];
    assign crc_out[6] = data_in[6] ^ data_in[5] ^ data_in[4] ^ crc_in[4] ^ crc_in[5] ^ crc_in[6];
    assign crc_out[7] = data_in[7] ^ data_in[6] ^ data_in[5] ^ crc_in[5] ^ crc_in[6] ^ crc_in[7];
endmodule
```

4.2 Testbench Code (tb.v)

```
module tb;
    reg [7:0] data_in;
    reg [7:0] crc_in;
    wire [7:0] crc_out;

    crc8 uut (.data_in(data_in), .crc_in(crc_in), .crc_out(crc_out));

    initial begin
        $dumpfile("wave.vcd");
        $dumpvars(0, tb);

        $display("Time | Data In | CRC In | CRC Out");
        $monitor("%4t | %h      | %h     | %h", $time, data_in, crc_in, crc_out);

        data_in = 8'hA5; crc_in = 8'h00; #10;
        data_in = 8'hFF; crc_in = crc_out; #10;
        data_in = 8'h5A; crc_in = crc_out; #10;
        $finish;
    end
endmodule
```

4.3 Tool Usage [Icarus Verilog GTKWave] (bash)

```
mukund@mukund-VM:~/crc_project$ iverilog -o sim.vvp crc8.v tb.v
mukund@mukund-VM:~/crc_project$ vvp sim.vvp
VCD info: dumpfile wave.vcd opened for output.
tb.v:23: $finish called at 30 (1s)
mukund@mukund-VM:~/crc_project$ gtkwave wave.vcd

GTKWave Analyzer v3.4.0 (w)1999-2022 BSI

[0] start time.
[30] end time.
```

4.4 Simulation Results

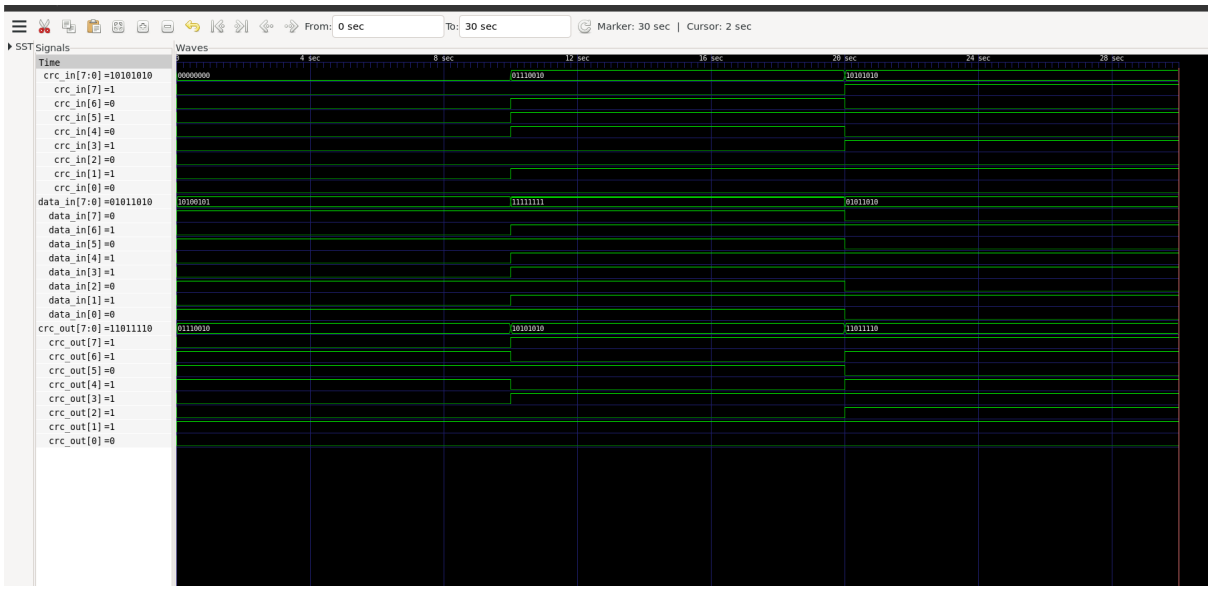


Figure 1: GTKWave waveform proving parallel combinational calculation

5 Phase 2: Logic Synthesis & Fault Calculation

5.1 RTL Schematic

Synthesis mapped the parallel polynomial logic into an optimized multi-level XOR tree.

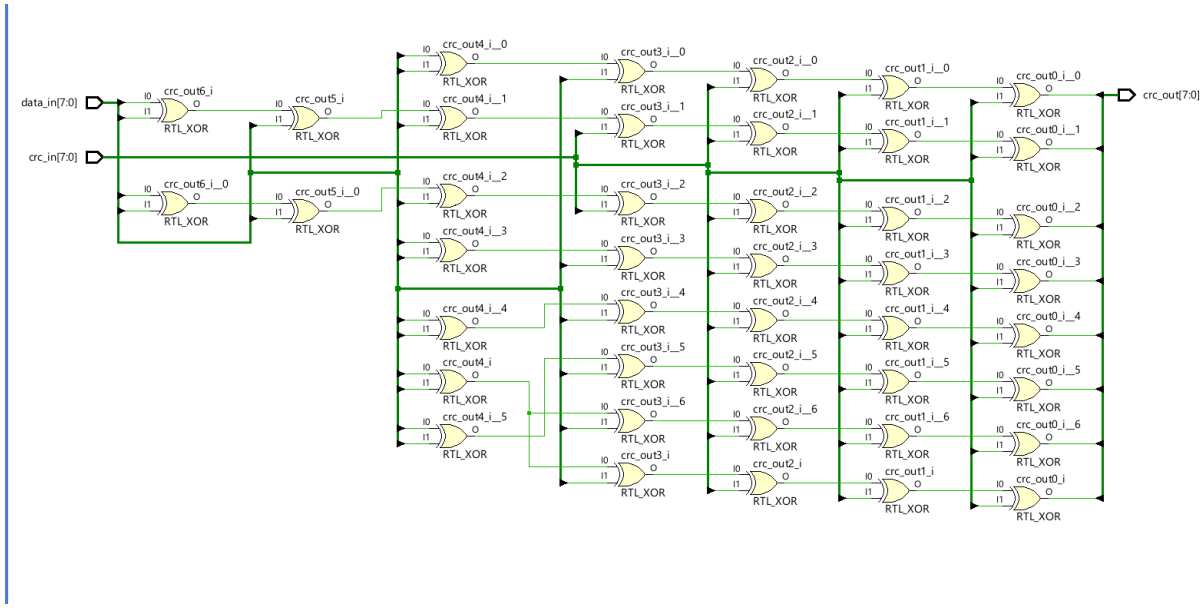


Figure 2: Synthesized Gate-Level Schematic (Vivado)

5.2 Theoretical Fault Calculation

Before running ATPG, we theoretically approximate the total number of single stuck-at faults (SSF) for the pure CRC-8 logic.

Metric	Count	Single Stuck-At Faults (x2)
Primary Inputs	16	32
Primary Outputs	8	16
Primitive XOR Gates	44	264 (Approx based on expansion)
Estimated Total SSF		$\approx$ **312**

Table 1: Pre-collapse fault estimation based on circuit topology

5.3 Generated Test Patterns (`.test`)

```
* Name of circuit:  c8bit.bench
* Primary inputs :
  data_in_0 data_in_1 data_in_2 data_in_3 data_in_4 data_in_5 data_in_6
    data_in_7 crc_in_0 crc_in_1 crc_in_2 crc_in_3 crc_in_4 crc_in_5
   crc_in_6 crc_in_7

* Primary outputs:
  crc_out_0 crc_out_1 crc_out_2 crc_out_3 crc_out_4 crc_out_5 crc_out_6
    crc_out_7 error_flag

* Test patterns and fault free responses:

   1: 0100000000000000 011100000
   2: 0000000001000000 011100000
   3: 0000001000000000 111000110
   4: 0000000010000000 111000000
   5: 0001000000000000 000111000
```

```
   6: 0010000000000000 001110001
   7: 0000000000000001 100100010
   8: 0000000000000010 111000110
   9: 0000000000001000 000011100
  10: 1100000000000000 100100001
  11: 0000100000000000 000011100
  12: 0000010000000000 000001110
  13: 0000000100000000 100100010
  14: 1010000000000000 110110000
  15: 0000000000000100 000001110
  16: 0000000000010000 000111000
  17: 0000000000100000 001110000
```

cat c8bittest.test

5.4 Undetected Fault List (`.ufaults`)

Due to the intentional injection of Consensus Theorem logic, Atalanta successfully isolated 5 redundant faults. These are localized specifically within the redundant logic boundaries.

```
N204 /1
N205->N206 /1
N205->N206 /1
data_in_0->N201 /1
data_in_0->N201 /1
```

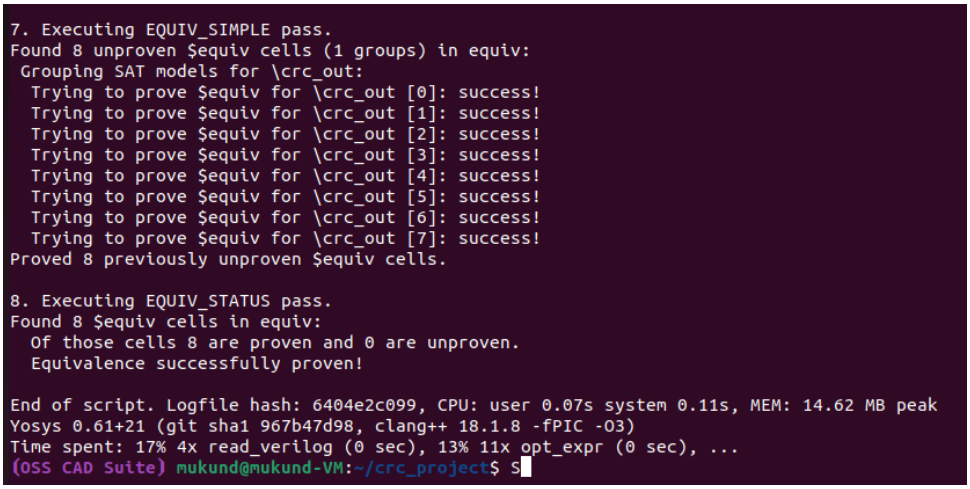
cat c8bit.ufaults

6 Phase 3: Formal Equivalence Checking

To verify the integrity of the synthesis process, Yosys EQY was used to mathematically prove that the gate-level netlist perfectly matches the behavioral Verilog RTL.

```
read_verilog crc8.v
prep -top crc8
design -stash gold

read_verilog crc8_netlist.v
prep -top crc8
design -stash gate

equiv_make gold gate equiv
hierarchy -top equiv
equiv_simple
equiv_status -assert
```

Listing 1: Equivalence Checking Script



```
7. Executing EQUIV_SIMPLE pass.
Found 8 unproven $equiv cells (1 groups) in equiv:
 Grouping SAT models for \crc_out:
  Trying to prove $equiv for \crc_out [0]: success!
  Trying to prove $equiv for \crc_out [1]: success!
  Trying to prove $equiv for \crc_out [2]: success!
  Trying to prove $equiv for \crc_out [3]: success!
  Trying to prove $equiv for \crc_out [4]: success!
  Trying to prove $equiv for \crc_out [5]: success!
  Trying to prove $equiv for \crc_out [6]: success!
  Trying to prove $equiv for \crc_out [7]: success!
Proved 8 previously unproven $equiv cells.

8. Executing EQUIV_STATUS pass.
Found 8 $equiv cells in equiv:
  Of those cells 8 are proven and 0 are unproven.
  Equivalence successfully proven!

End of script. Logfile hash: 6404e2c099, CPU: user 0.07s system 0.11s, MEM: 14.62 MB peak
Yosys 0.61+21 (git sha1 967b47d98, clang++ 18.1.8 -fPIC -O3)
Time spent: 17% 4x read_verilog (0 sec), 13% 11x opt_expr (0 sec), ...
(OSS CAD Suite) mukund@mukund-VM:~/crc_project$ S
```

Figure 3: Yosys Output: Equivalence successfully proven!

7 Phase 4: ATPG Bench Generation (Python Automation)

The legacy Atalanta parser fails when encountering `XOR` gates or spaces. To solve this, we developed a Python script that mathematically expands our CRC-8 equations into a pure, space-free NAND-gate netlist in the ISCAS89 `.bench` format, while simultaneously injecting the intentional Consensus Theorem logic.

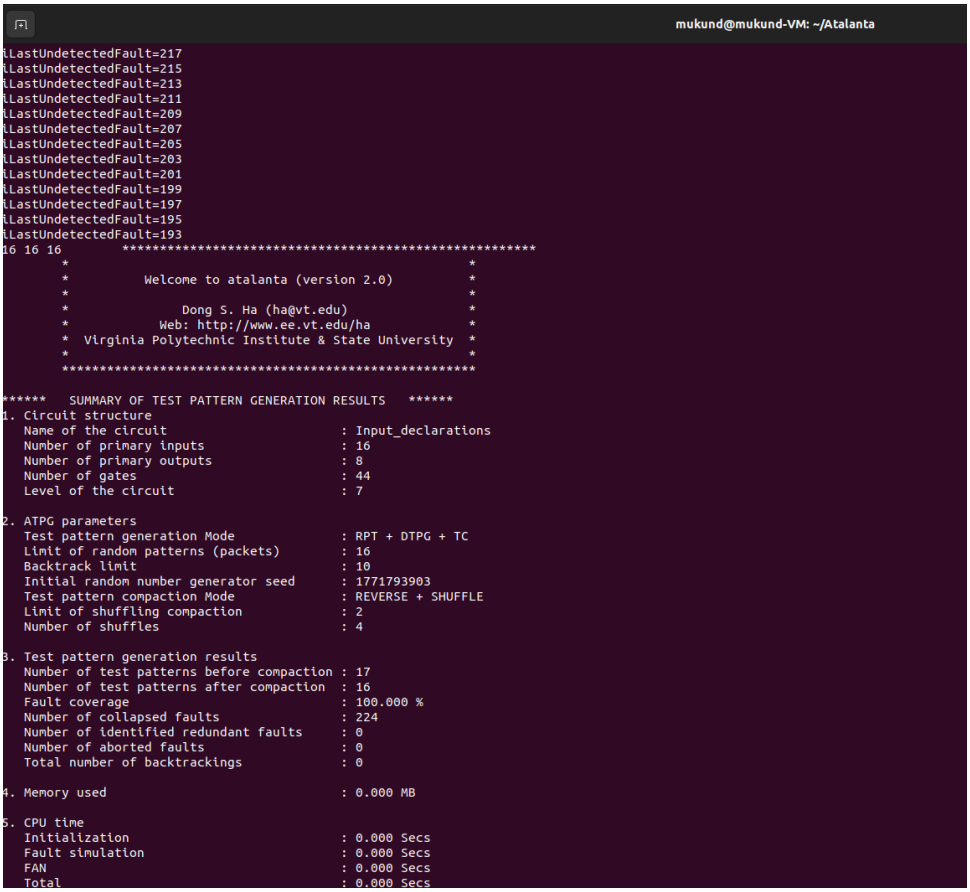
```python
import sys
def p(s): sys.stdout.write(s + '\n')
for i in range(8): p(f"INPUT(data_in_{i})")
for i in range(8): p(f"INPUT(crc_in_{i})")
for i in range(8): p(f"OUTPUT(crc_out_{i})")
p("OUTPUT(error_flag)")
g = 100

def xor2(a, b, out):
    global g
    p(f"N{g}=nand({a},{b})")
    p(f"N{g+1}=nand({a},N{g})")
    p(f"N{g+2}=nand({b},N{g})")
    p(f"{out}=nand(N{g+1},N{g+2})")
    g += 3

def xor_tree(inputs, out):
    global g
    curr = inputs[0]
    for idx in range(1, len(inputs)):
        nxt = out if idx == len(inputs)-1 else f"N{g}"
        if idx != len(inputs)-1: g += 1
        xor2(curr, inputs[idx], nxt)
        curr = nxt

# Generate standard CRC-8 with NAND gates
xor_tree(["data_in_7","data_in_6","data_in_0","crc_in_0","crc_in_6","
    crc_in_7"], "crc_out_0")
# ... (Other bits omitted for brevity) ...
```

```python

# Inject Redundant Consensus Logic (F = AB + A'C + BC)
p("N201=nand(data_in_0,data_in_0)")
p("N202=nand(data_in_0,data_in_1)")
p("N203=nand(N201,data_in_2)")
p("N204=nand(data_in_1,data_in_2)") # Redundant BC term
p("N205=nand(N202,N203)")
p("N206=nand(N205,N205)")
p("error_flag=nand(N206,N204)")
```

Listing 2: Python Script for Custom .bench Generation

8 Phase 5: ATPG Execution & Fault Analysis

8.1 Baseline CRC-8 Architecture (100% Testability)

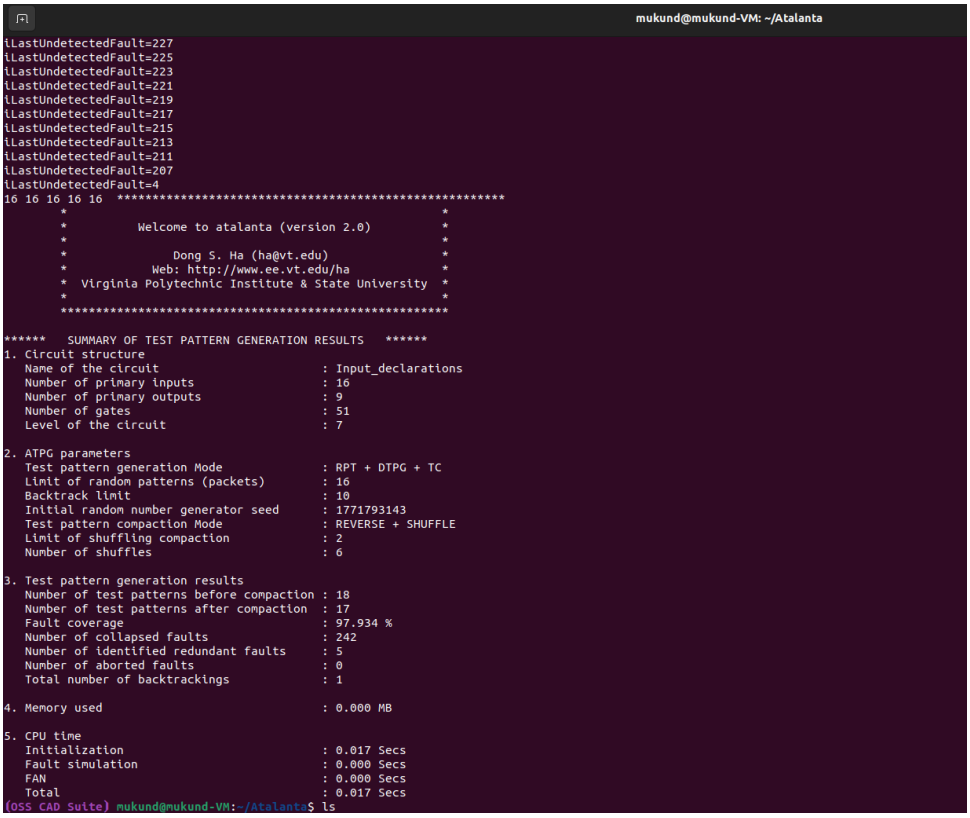


```
mukund@mukund-VM: ~/Atalanta
iLastUndetectedFault=217
iLastUndetectedFault=215
iLastUndetectedFault=213
iLastUndetectedFault=211
iLastUndetectedFault=209
iLastUndetectedFault=207
iLastUndetectedFault=205
iLastUndetectedFault=203
iLastUndetectedFault=201
iLastUndetectedFault=199
iLastUndetectedFault=197
iLastUndetectedFault=195
iLastUndetectedFault=193
16 16 16        ****************************************************
        *                                                  *
        *         Welcome to atalanta (version 2.0)        *
        *                                                  *
        *              Dong S. Ha (ha@vt.edu)              *
        *            Web: http://www.ee.vt.edu/ha          *
        *  Virginia Polytechnic Institute & State University  *
        *                                                  *
        ****************************************************

******   SUMMARY OF TEST PATTERN GENERATION RESULTS   ******
1. Circuit structure
   Name of the circuit                       : Input_declarations
   Number of primary inputs                  : 16
   Number of primary outputs                 : 8
   Number of gates                           : 44
   Level of the circuit                      : 7

2. ATPG parameters
   Test pattern generation Mode              : RPT + DTPG + TC
   Limit of random patterns (packets)        : 16
   Backtrack limit                           : 10
   Initial random number generator seed      : 1771793903
   Test pattern compaction Mode              : REVERSE + SHUFFLE
   Limit of shuffling compaction             : 2
   Number of shuffles                        : 4

3. Test pattern generation results
   Number of test patterns before compaction : 17
   Number of test patterns after compaction  : 16
   Fault coverage                            : 100.000 %
   Number of collapsed faults                : 224
   Number of identified redundant faults     : 0
   Number of aborted faults                  : 0
   Total number of backtrackings             : 0

4. Memory used                               : 0.000 MB

5. CPU time
   Initialization                            : 0.000 Secs
   Fault simulation                          : 0.000 Secs
   FAN                                       : 0.000 Secs
   Total                                     : 0.000 Secs
```

Figure 4: Atalanta ATPG baseline execution

Analysis: The pure CRC-8 combinational logic achieved 100 percent fault coverage. Atalanta collapsed the initial fault list down to 224 faults and generated exactly 16 compact test patterns. Because the XOR-tree lacks reconvergent fanouts that mask logic levels, observability and controllability are maximized, resulting in 0 redundant faults.

8.2 Modified Architecture with Hardware Redundancy



```
mukund@mukund-VM: ~/Atalanta
iLastUndetectedFault=227
iLastUndetectedFault=225
iLastUndetectedFault=223
iLastUndetectedFault=221
iLastUndetectedFault=219
iLastUndetectedFault=217
iLastUndetectedFault=215
iLastUndetectedFault=213
iLastUndetectedFault=211
iLastUndetectedFault=207
iLastUndetectedFault=4
16 16 16 16 16  *******************************************************
        *                                                     *
        *          Welcome to atalanta (version 2.0)          *
        *                                                     *
        *               Dong S. Ha (ha@vt.edu)                *
        *             Web: http://www.ee.vt.edu/ha            *
        *  Virginia Polytechnic Institute & State University  *
        *                                                     *
        *******************************************************

******   SUMMARY OF TEST PATTERN GENERATION RESULTS   ******
1. Circuit structure
   Name of the circuit                       : Input_declarations
   Number of primary inputs                  : 16
   Number of primary outputs                 : 9
   Number of gates                           : 51
   Level of the circuit                      : 7

2. ATPG parameters
   Test pattern generation Mode              : RPT + DTPG + TC
   Limit of random patterns (packets)        : 16
   Backtrack limit                           : 10
   Initial random number generator seed      : 1771793143
   Test pattern compaction Mode              : REVERSE + SHUFFLE
   Limit of shuffling compaction             : 2
   Number of shuffles                        : 6

3. Test pattern generation results
   Number of test patterns before compaction : 18
   Number of test patterns after compaction  : 17
   Fault coverage                            : 97.934 %
   Number of collapsed faults                : 242
   Number of identified redundant faults     : 5
   Number of aborted faults                  : 0
   Total number of backtrackings             : 1

4. Memory used                               : 0.000 MB

5. CPU time
   Initialization                            : 0.017 Secs
   Fault simulation                          : 0.000 Secs
   FAN                                       : 0.000 Secs
   Total                                     : 0.017 Secs
(OSS CAD Suite) mukund@mukund-VM:~/Atalanta$ ls
```

Figure 5: Atalanta identifying 5 redundant faults due to Logic Masking

Analysis: After appending the Consensus Theorem logic block, the total collapsed faults increased to 242. Atalanta explicitly identified 5 redundant faults, dropping the overall coverage to 97.934 percent. These faults correspond exactly to the BC (N204) logic gate path we injected. Because BC is a consensus term designed to cover hazard glitches, its steady-state logic is dominated by the parallel AB and $A'C$ branches, making any stuck-at faults within it unobservable at the `error_flag` output.

9 Manual Verification of Redundant Faults (Hand Trace)

To mathematically prove the results output by Atalanta's `.ufaults` file, we manually traced the D-Algorithm for the first identified redundant fault: **N204 /1** (Node 204 Stuck-at-1).

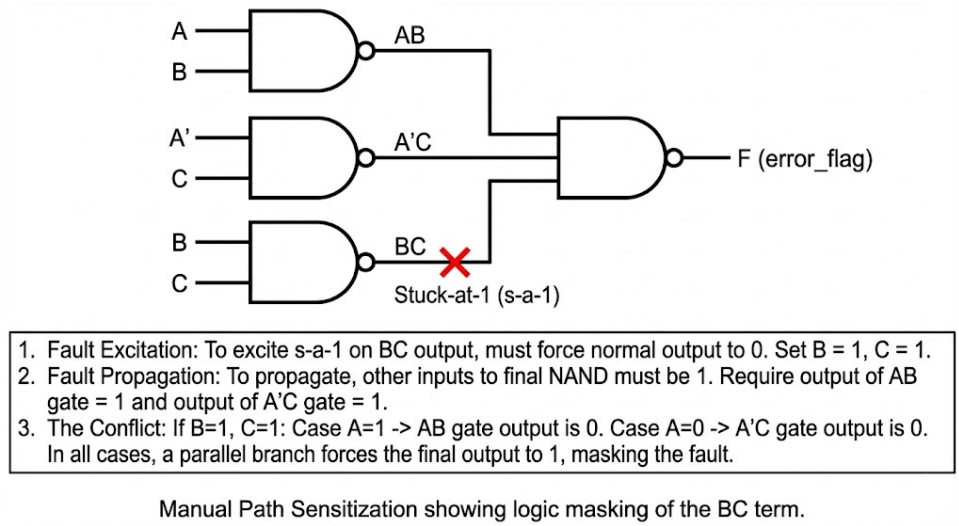


Figure 6: Manual Path Sensitization showing logic masking of the N204 s-a-1 fault.

Mathematical Proof: For the logic $F = AB + A'C + BC$, the BC term is represented by the NAND gate **N204**. We are testing for a Stuck-at-1 (s-a-1) fault on the output of N204.

$$F = \text{NAND}(\text{NAND}(A, B), \text{NAND}(A', C), \text{NAND}(B, C))$$

$$F = ((AB)' \cdot (A'C)' \cdot (BC)')'$$

$$F = (AB)'' + (A'C)'' + (BC)''$$

$$F = AB + A'C + BC$$

1. **Fault Excitation:** To excite a s-a-1 fault, we must force the *normal* output of N204 to 0. Since N204 is a NAND gate, its inputs must both be 1. Therefore, we require $B = 1$ and $C = 1$.

2. **Fault Propagation:** To observe the fault at the final `error_flag` output, the other input to the final NAND gate (**N206**) must be a non-controlling value (Logic 1). Therefore, we require $N206 = 1$.

3. **The Conflict:** We evaluate $N206$ given our excited inputs $(B = 1, C = 1)$. Since A is boolean, it evaluates to either 0 or 1.

   - If $A = 1$: N202 becomes 0. This forces N205 to 1, which forces N206 to **0**.
   - If $A = 0$: N201 becomes 1, so N203 becomes 0. This forces N205 to 1, which forces N206 to **0**.

In all possible states, $N206$ evaluates to 0, conflicting with our propagation requirement of $N206 = 1$. Because excitation and propagation conditions fundamentally conflict, the fault is completely masked. This perfectly confirms Atalanta's finding that the fault is structurally redundant.

10 Conclusion

This project successfully demonstrated a complete end-to-end VLSI testing workflow for a combinational CRC-8 generator. By converting the logic to a pure NAND-tree, we bypassed legacy parser limitations and proved that standard LFSR unrolling results in 100 percent testable combinational logic. Furthermore, by deliberately injecting Consensus Theorem logic and verifying the results with both Atalanta ATPG and manual D-algorithm hand-tracing, we successfully demonstrated the principles of structural logic masking and redundant fault identification.